

DS	Session	Stack	Question Information	Level 1	Challenge 50
----	---------	-------	----------------------	---------	--------------

Question description

Hassan gets a job in a software company in Hyderabad. The training period for the first three months is 20000 salary. Then incremented to 25000 salaries.

Training is great but they will give you a programming task every day in three months. Hassan must finish it in the allotted time. His teammate Jocelyn gives him a task to complete the concept of Postfix to Prefix Conversion for a given expression. can you help him?

Functional Description:

- Read the Prefix expression in reverse order (from right to left)
- If the symbol is an operand, then push it onto the Stack
- If the symbol is an operator, then pop two operands from the Stack
 - Create a string by concatenating the two operands and the operator after them.
 - string = operand1 + operand2 + operator**
 - And push the resultant string back to Stack
- Repeat the above steps until end of Prefix expression.

Constraints

the input should be a expressions

Input Format

Single line represents the postfix expressions

Output Format

Single line represents the prefix expression

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

ABC/-AK/L-*

EXPECTED OUTPUT

*-A/BC-/AKL

Test Case 2

INPUT (STDIN)

AB+CD-*

EXPECTED OUTPUT

*+AB-CD

▼ Mandatory Test Cases

Test Case 1

KEYWORD

string postToPre(string
post_exp)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

5

Test Case 2

TOKEN COUNT

220

Test Case 3

NLOC

46

Stack10.c +

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4 #include <stdlib.h>
5
6 #define MAX 1000
7
8 typedef struct {
9     char str[MAX];
10 } StackItem;
11
12 StackItem stack[MAX];
13 int top = -1;
14
15 void push(char *s) {
16     top++;
17     strcpy(stack[top].str, s);
18 }
19
20 void pop(char *s) {
21     if (top >= 0) {
22         strcpy(s, stack[top].str);
23         top--;
24     }
25 }
```

```
26
27int isOperator(char c) {
28    return (c == '+' || c == '-' || c == '*' || c == '/' || c == '^');
29}
30
31void postToPre(char *post_exp, char *result) {
32    top = -1;
33    int len = strlen(post_exp);
34
35    for (int i = 0; i < len; i++) {
36        char c = post_exp[i];
37        if (!isOperator(c)) {
38            char temp[2] = {c, '\0'};
39            push(temp);
40        } else {
```

```
41         if (top >= 1) {
42             char op2[MAX], op1[MAX];
43             pop(op2);
44             pop(op1);
45
46             char combined[MAX];
47             combined[0] = c;
48             combined[1] = '\0';
49             strcat(combined, op1);
50             strcat(combined, op2);
51
52             push(combined);
53         }
54     }
55 }
```

main | CH.SC.U4CSE25225 | Stack10.c | Ln 83, Col 1 | Spaces: 4 | UTF-8 | CRLF | C

```
56
57     while (top > 0) {
58         char op2[MAX], op1[MAX];
59         pop(op2);
60         pop(op1);
61         char combined[MAX];
62         strcpy(combined, op1);
63         strcat(combined, op2);
64         push(combined);
65     }
66
67     if (top >= 0) {
68         strcpy(result, stack[top].str);
69     } else {
```

```
70     result[0] = '\0';
71 }
72}
73
74int main() {
75     char post_exp[MAX];
76     if (scanf("%s", post_exp) != 1) return 0;
77
78     char pre_exp[MAX];
79     postToPre(post_exp, pre_exp);
80     printf("%s\n", pre_exp);
81     return 0;
82}
83
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
ABC/-AK/L-*
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack10.exe
*-A/BC-/AKL
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
AB+CD-*
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack10.exe
*+AB-CD
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```